NGUYEN DEVELOPMENT · PHPDBADMIN

Terms of Reference — phpdbadmin

Pre-Project: Terms of Reference mit Ausgangslage und Rückfragen und Ideen-Backlog

Phase	01 Pre-Project
Eingang	CR-001 phpdbadmin (Fassung 3), Feasibility v1.0
Träger	Nguyen Development (Einzelfirma), Hoang Nguyen · Lizenz MIT
Vorgehen	DSDM/AgilePM — Dokumentation in dokumentationen/, ein PDF je Phase
Stand	28.10.2026, 15:47 · baut auf Commit abf0697 + Änderungen dieses Standes auf
Quelle	dokumentationen/ im Repo — dieses PDF wird bei jeder Änderung neu gebaut

Dieses Dossier enthält

1. Terms of Reference — phpdbadmin
2. Anhang A — 01 — Pre-Project
3. Anhang B — Ausgangslage und Rückfragen — phpdbadmin
4. Anhang C — Ideen-Backlog — phpdbadmin

Inhalt

Terms of Reference — phpdbadmin

1. Projektziel
2. Hintergrund
3. Systemkontext
4. Rahmen
5. Umfang
6. Priorisierung: Stufen statt Termine
7. Fahrplan und Meilensteine
8. Erfolgskriterien
9. Risiken
10. Entscheide und Vorbedingungen
11. DSDM-Prinzipien-Check

Anhang A — 01 — Pre-Project

Anhang B — Ausgangslage und Rückfragen — phpdbadmin

1. Was feststeht
2. Was ich selbst nachgelesen habe (keine Rückfrage nötig)
3. Rückfragen
4. Entscheide aus der interaktiven Runde vom 28.10.2026

Anhang C — Ideen-Backlog — phpdbadmin

1. Anmeldung und Konten
2. Verbindungen
3. Bedienung
4. Daten
5. Struktur und SQL
6. Export, Import, Betrieb

Architektur-Hinweis

Terms of Reference — phpdbadmin

Feld	Wert
Status	🟡 **v1.0 zur Verabschiedung** (28.10.2026) — nach CR-001 Fassung 3 (Weiterführen) und Feasibility v1.0 (6 bestanden, 4 bedingt); verabschiedet, sobald Hoang zustimmt
Eingang	CR-001 Fassung 3 (Weiterführen 28.10.2026), Feasibility v1.0, Ausgangslage und Rückfragen, Ideen-Backlog
Ausgang	Foundation: Gestaltung, PRL, Solution-Architektur, Datenmodell, Timebox-Plan, ADR-001 bis ADR-010
Träger	Nguyen Development, Einzelfirma von Hoang Nguyen, Zürich (entschieden 28.10.2026, R-01)

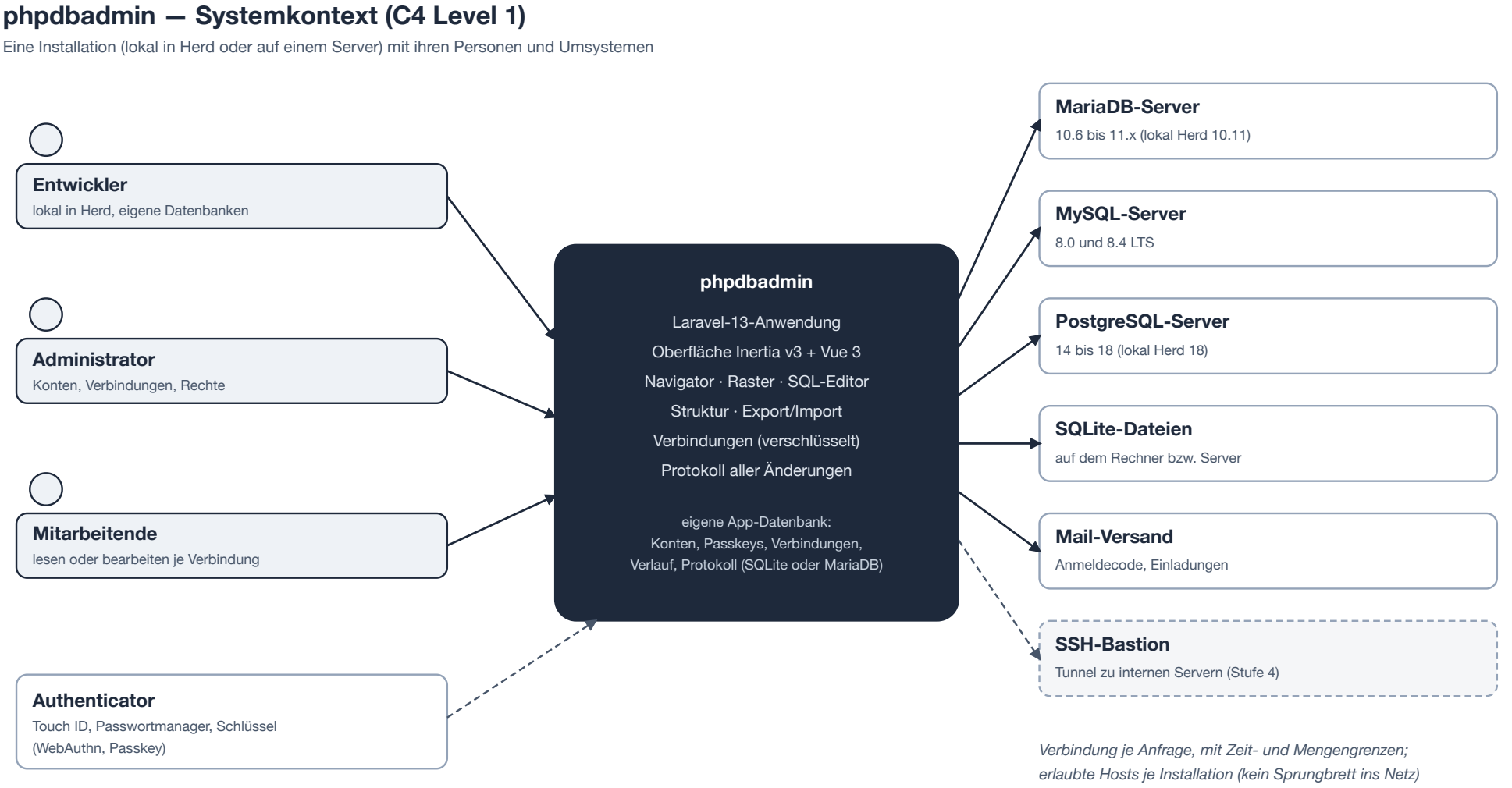
1. Projektziel

Nguyen Development baut **phpdbadmin**, ein Werkzeug im Browser, mit dem man **MariaDB, MySQL, PostgreSQL und SQLite mit derselben Bedienung** ansieht, bearbeitet, strukturiert und austauscht — geführt (Vorschau, Sammeln, Rückgängig), sicher (Konten ohne Passwort, Rechte, Schreibschutz, Protokoll) und schnell, auf Laravel 13, lokal in Herd und selbst gehostet auf jedem PHP-Server. v1.0 ersetzt phpMyAdmin und phpPgAdmin im Alltag von Hoang.

2. Hintergrund

Die PHP-Klassiker stehen technisch still (phpMyAdmin 5.2 nicht für PHP 8.4+ freigegeben, phpPgAdmin seit 2023 ohne Commit), Adminer ist breit, aber knapp, und die modernen Werkzeuge sind nicht PHP oder nicht im Browser (Marktbild). Laravel 13 bringt Verbindungen zur Laufzeit, Introspektion für alle Treiber und DDL mit Vorschau bereits mit; die passwortlose Anmeldung und die Tests auf mehreren Datenbanken sind aus Basalto erprobt. Die Feasibility hat alle zehn Prüfpunkte mit Proben auf MySQL 8.4, MariaDB 10.11, PostgreSQL 18 und SQLite belegt: sechs bestanden, vier bedingt — die Auflagen (Anwenden je Treiber, SQLite im Arbeitsprozess, Kontowarnung, maskierte Meldungen, Name absichern) stehen als Anforderungen in CR-001.

3. Systemkontext



Eine Installation mit ihren Personen (Entwickler, Administrator, Mitarbeitende, Authenticator) und Umsystemen (MariaDB-, MySQL- und PostgreSQL-Server, SQLite-Dateien, Mail, später SSH-Bastion). Details: CR-001 § 3.2.

4. Rahmen

Aspekt	Festlegung
Träger	Nguyen Development (R-01)
Business Sponsor, Visionary, Projektleitung, Technical Coordinator	Hoang Nguyen in Personalunion (R-02)
Team	Hoang Nguyen + Claude Code (Pair); Rückfragen interaktiv mit Empfehlung zuerst, Defaults ohne Rückfrage
Business Ambassador	Hoang als erster Nutzer; Gegenleser ab Alpha gesucht
Vorrang	Basalto bis zur Beta vor phpdbadmin; phpdbadmin in kleinen Timeboxes daneben (entschieden, R-03)
Zeitmodell	Stufen statt Termine; rund 42–53 PT bis v1.2 (nach Feasibility)
Technischer Rahmen	Laravel 13.39 (PHP 8.5), Inertia v3, Vue 3, TypeScript, Tailwind 4, Wayfinder, Fortify nur für Passkeys; App-Datenbank SQLite; CodeMirror 6 und TanStack Virtual (probeweise installiert, ADR-005/006); Herd lokal mit MySQL 8.4, MariaDB 10.11, PostgreSQL 18
Starter-Kit	Passwort, Dashboard und Profil werden entfernt (Vorgabe Hoang 28.10.2026)
Betriebsmodell	lokal und selbst gehostet, kein SaaS (entschieden, R-11)
Lizenz	Open Source unter MIT (entschieden, R-40); Abhängigkeiten MIT-verträglich
Name	phpdbadmin, endgültig (entschieden, R-04); Markenrecherche vor dem ersten öffentlichen Auftritt (F-10)
Sprache	Oberfläche Deutsch und Englisch (Sie-Form); Bezeichner englisch; Doku Deutsch mit echten Umlauten
Doku	DSDM in `dokumentationen/`, bei jedem Commit nachgeführt; ein PDF je Phase; Diagramme nur aus `werkzeuge/diagramme.py`; Vault-Projektnotiz

5. Umfang

5.1 Enthalten

	Stufe 0–2 — v1.0	Stufe 3 — v1.1	Stufe 4 — v1.2
Zugang	Passkey, Code per Mail, Einladung, Rollen und Rechte je Verbindung, Protokoll	—	Sitzungen verwalten
Verbindungen	vier Treiber, testen, verschlüsselt, Umgebung mit Schreibschutz, erlaubte Hosts	—	Herd-Erkennung, SSH-Tunnel
Arbeiten	Navigator, Befehlspalette, Reiter, Raster, Filter, Bearbeiten mit Vorschau und Rückgängig, Struktur ansehen und ändern, SQL-Editor mit Grenzen	Views, Routinen, Trigger, Sequenzen, Beziehungsdiagramm, EXPLAIN	Schemavergleich, Laravel-Migration, KI optional
Austausch	Export CSV, JSON, SQL; Import CSV und SQL	Sicherung und Wiederherstellung, Excel	—
Server	—	Übersicht, Prozesse, Variablen, Benutzer und Rechte, Wartung	—

5.2 Nicht enthalten

SQL Server und Oracle (vorerst) · NoSQL · SaaS mit fremden Zugangsdaten · Desktop-App · Berichte und BI · Replikation und Cluster · Hosting-Panel · Passwort-Anmeldung, Dashboard, Profilseite.

6. Priorisierung: Stufen statt Termine

1. **Stufe 0 — Fundament:** Starter-Kit bereinigt (Passwort, Dashboard, Profil weg), Passkey und Code, Verbindungen, Dialekt-Schicht, Tests auf vier Datenbanken
2. **Stufe 1 — Lesen:** Navigator, Raster, Filter, Struktur ansehen, SQL-Editor, Befehlspalette → **Alpha**
3. **Stufe 2 — Bearbeiten:** Bearbeiten mit Vorschau und Rückgängig, Struktur ändern, Export und Import, Rechte, Protokoll → **v1.0**
4. **Stufe 3 — Verwalten:** Server-Verwaltung, weitere Objekte, Beziehungsdiagramm, EXPLAIN, Sicherung → **v1.1**
5. **Stufe 4 — Komfort und Team:** Schemavergleich, Laravel-Migration, Herd, SSH-Tunnel, KI optional → **v1.2**

Der Must-Anteil bis v1.0 liegt nach CR-001 Fassung 3 bei 68 % (62 Anforderungen, davon 42 Must); die PRL bringt ihn unter 60 %.

7. Fahrplan und Meilensteine

phpdbadmin — Stufen und Meilensteine

Reihenfolge, ohne Termine. Jede Stufe ist für sich nutzbar; phpdbadmin läuft neben Basalto, das bis zur Beta Vorrang hat (R-03)



Summe rund 40–50 Personentage bis v1.2 (Schätzung vor Feasibility). Rückfallposition: nach Stufe 2 ist das Werkzeug vollständig nutzbar — Verwalten und Team folgen.

Meilenstein	Ergebnis	Abnahme
Alpha (nach Stufe 1)	lesend im eigenen Alltag auf allen vier Datenbanken	Hoang nutzt es eine Woche statt phpMyAdmin/TablePlus zum Nachsehen; Tests auf vier Treibern grün
v1.0 (nach Stufe 2)	ersetzt phpMyAdmin und phpPgAdmin	Kernabläufe auf vier Treibern getestet; White-Box-Sicherheitsprüfung ohne hohen Befund; WCAG 2.2 AA in den Kernbildschirmen
v1.1 (nach Stufe 3)	Server-Verwaltung vollständig	Benutzer und Rechte auf drei Server-Treibern getestet
v1.2 (nach Stufe 4)	Team und Laravel	Schemavergleich und Migration auf zwei Treibern gegenseitig

Rückfallposition: nach Stufe 2 ist das Werkzeug vollständig nutzbar; Verwalten und Team folgen bei Gelegenheit.

8. Erfolgskriterien

Hoang ersetzt phpMyAdmin, phpPgAdmin und den Desktop-Client im Alltag ab v1.0 · häufige Aufgaben in höchstens drei Klicks oder zwei Tastenkürzeln · dieselben Abläufe automatisch auf vier Datenbanken getestet · erste Seite einer Tabelle mit einer Million Zeilen lokal unter einer Sekunde · Sicherheitsprüfung ohne hohen Befund · WCAG 2.2 AA in den Kernbildschirmen.

9. Risiken

Aus CR-001 § 10: Zeit neben Basalto und Fristo (R1) · Treiberunterschiede (R2) · Sicherheit eines Werkzeugs mit Zugang zu allen Datenbanken (R3) · Leistung des Rasters (R4) · Mitbewerber Jaxon DbAdmin und phpMyAdmin 6 (R5) · Name (R6) · Arbeitsvertrag (R7). Durch die Feasibility gesenkt: R2 (Dialekt-Unterschiede gemessen und eingegrenzt), R4 (erste Seite 0,1–0,6 ms in der Datenbank). Neu sichtbar: Datenverlust beim SQLite-Umbau in einer Transaktion und stille Abbrüche bei MySQL — beides mit Anforderung (N-03, Q-02) und Probe belegt.

10. Entscheide und Vorbedingungen

Reihe	Wo	Stand
R-01 bis R-42	Rückfragen-Katalog	erste Runde 28.10.2026: R-01, R-03, R-04, R-11, R-40 entschieden, übrige mit Annahme
Starter-Kit ohne Passwort, Dashboard, Profil	Hoang 28.10.2026	entschieden
CR-001 Weiterführen	Hoang 28.10.2026 («Feasibility starten»)	entschieden
Feasibility F-1 bis F-10, Pakete CodeMirror und TanStack probeweise, MySQL 8.4 in Herd	Feasibility, Freigaben Hoang 28.10.2026	abgeschlossen 28.10.2026

Vorbedingungen vor Stufe 0: Entscheid «Weiterführen» auf CR-001 ✓ · Feasibility mit F-1 bis F-10 ✓ · MySQL in Herd ✓ · Terms of Reference verabschiedet (Hoang) · Foundation (Gestaltung, PRL, Architektur, Datenmodell, Timebox-Plan, ADRs). **Vor dem ersten öffentlichen Auftritt:** Auflagen aus Name und Marke: Domains (`.ch`, nach Wahl `.org`, `.net`, `.io`, `.dev`) gesichert, Repo `hoang-79/phpdbadmin` (Organisation `phpdbadmin` ist belegt), Hinweis «nicht verbunden mit phpMyAdmin» · Lizenzdatei MIT im Repo · Nebenbeschäftigung gemeldet (R-05).

11. DSDM-Prinzipien-Check

Prinzip	Wie eingelöst
Geschäftsnutzen zuerst	Alpha und v1.0 lösen Hoangs eigenes Problem zuerst; jede Stufe ist für sich nutzbar
Rechtzeitig liefern	Stufen statt Termine; Alpha früh als nutzbarer Zwischenstand; Rückfallposition nach Stufe 2
Zusammenarbeiten	Hoang + Claude im Pair; Rückfragen interaktiv mit Empfehlung; Gegenleser ab Alpha
Qualität nie opfern	Tests gegen vier Datenbanken, Vertragstests der Dialekt-Schicht, Sicherheitsprüfung, WCAG, Kontraste gerechnet
Inkrementell auf festem Fundament bauen	Feasibility-Proben auf allen vier Treibern, Stufe 0 Fundament vor jeder Funktion
Iterativ entwickeln	Timeboxes mit Bedienkonzept vorher, Durchlauf und Abnahme nachher, Protokoll je Timebox
Kontinuierlich und klar kommunizieren	Doku bei jedem Commit, ein PDF je Phase, Vault-Notiz
Kontrolle zeigen	Rückfragen-Katalog, PRL mit Must-Anteil, Timebox-Protokolle, ADRs

Anhang A — 01 — Pre-Project

> **PDF der Phase:** terms-of-reference.pdf — bei jeder Änderung neu gebaut (`python3 dokumentationen/werkzeuge/pdf-bauen.py --nur 01`).

Projekteingang ist CR-001. Das Pre-Project klärt mit Hoang, was feststeht und was offen ist, sammelt die Ideen und hält den Auftrag in den Terms of Reference fest.

Dokument	Inhalt	Status
Ausgangslage und Rückfragen	Was feststeht, was nachgelesen ist, **Interview R-01 bis R-42** mit Annahmen	🔨 erste Runde 28.10.2026 — R-01, R-03, R-04, R-11, R-40 entschieden
Ideen-Backlog	Alle Ideen mit Nutzen, Herkunft und MoSCoW-Vorschlag	🔨 erste Sammlung
Terms of Reference	Projektziel, Hintergrund, Systemkontext, Rahmen, Umfang, Priorisierung, Fahrplan, Erfolgskriterien, Risiken, Vorbedingungen, DSDM-Check	🟡 v1.0 zur Verabschiedung (28.10.2026, nach Feasibility)

Diagramme (`diagrams/`): Anmeldung ohne Passwort (Code und Passkey, Entferntes aus dem Starter-Kit). Systemkontext und Fahrplan liegen in `00-change_request/diagrams/`.

Entschieden am 28.10.2026 (Hoang): Admin-Werkzeug für MariaDB, MySQL, PostgreSQL und SQLite auf Laravel · Starter-Kit ohne Passwort (nur Passkey und Code), ohne Dashboard, ohne Profil · Doku nach DSDM wie Fristo und Basalto · Träger Nguyen Development, Lizenz MIT (R-01, R-40) · lokal und selbst gehostet, kein SaaS (R-11) · neben Basalto, das bis zur Beta Vorrang hat (R-03) · Name phpdbadmin endgültig (R-04).

Anhang B — Ausgangslage und Rückfragen — phpdbadmin

Stand: 28.10.2026, erste Runde mit interaktiven Antworten zu R-01, R-03, R-04, R-11 und R-40 (Abschnitt 4). Dieses Dokument sammelt, was feststeht, was ich selbst nachgelesen habe und was nur Hoang beantworten kann. Jede Rückfrage trägt eine **Annahme**, die gilt, solange nichts dagegen spricht. Die wichtigsten Fragen werden interaktiv gestellt (Optionen zum Anklicken, Empfehlung zuerst); beantwortete Fragen bekommen das Datum der Antwort. Aus den Antworten entsteht die nächste Fassung von CR-001.

1. Was feststeht

Thema	Festlegung	Quelle
Vorhaben	Admin-Werkzeug für **MariaDB, MySQL, PostgreSQL und SQLite** wie phpMyAdmin oder phpPgAdmin, aber **intuitiver und benutzerfreundlicher**, für alle vier Plattformen	Hoang 28.10.
Rahmen	**Laravel-Projekt** im Repo `~/Development/phpdbadmin`, Herd-Site `phpdbadmin.test`	Hoang 28.10.
Starter-Kit	**Passwort entfernen** — nur Passkey und Code per Mail («Magic Code») · **Dashboard entfernen** · **Profil entfernen**	Hoang 28.10.
Dokumentation	in `dokumentationen/` nach DSDM wie Fristo und Basalto; Beginn mit dem Pre-Project	Hoang 28.10.
Sprache	Deutsch mit echten Umlauten in Doku, Kommentaren und Commits	globale Regeln
Arbeitsweise	Rückfragen interaktiv mit Empfehlung zuerst; Defaults ohne Rückfrage, nachfragen nur bei hohem Blast-Radius	globale Regeln, Lektionen
GitHub Actions	sparsam: ein schlanker Auftrag je Push, Schweres wöchentlich oder von Hand, Doku ohne Lauf	globale Regeln (25.10.2026)
Träger und Lizenz	**Produkt von Nguyen Development, Open Source unter MIT**	R-01, R-40
Betriebsmodell	**lokal (Herd) und selbst gehostet** auf eigenen Servern, mehrere Konten mit Rechten; **kein SaaS**	R-11
Vorrang	**neben Basalto** in kleinen Timeboxes; Basalto behält Vorrang bis zur Beta; Stufen 0–1 zuerst	R-03
Name	**phpdbadmin**, endgültig	R-04

2. Was ich selbst nachgelesen habe (keine Rückfrage nötig)

Frage	Antwort	Quelle
Was bringt das Starter-Kit mit?	Routen `home`, `dashboard` (auth + verified), `settings/profile`, `settings/security` (mit `RequirePassword`), `settings/password`, `settings/appearance`, `.well-known/passkey-endpoints`; Seiten `Welcome`, `Dashboard`, `auth/*` (Login, Register, ForgotPassword, ResetPassword, ConfirmPassword, TwoFactorChallenge, VerifyEmail), `settings/*` (Profile, Security, Appearance); Fortify mit Registrierung, Passwort-Reset, E-Mail-Bestätigung, Zwei-Faktor und **Passkeys** (`confirmPassword: true`) — gleiche Lage wie Basalto vor TB 00	`routes/web.php`, `routes/settings.php`, `resources/js/pages`, `config/fortify.php`
Welche Versionen sind installiert?	Laravel 13.39.0 · Fortify 1.41.0 (bringt `laravel/passkeys` mit) · Inertia Laravel 3.5.2 · Wayfinder 0.1.23 · PHPUnit 12.5.42 · Larastan 3.13.2 · Pint 1.32.1 · Boost 2.10.3; npm: `@inertiajs/vue3` 3, `@laravel/passkeys` 0.2, Vue 3.5, Tailwind 4, `vite-plus` 1.4.0	`composer show --direct`, `package.json`
Welche Datenbanken laufen lokal?	Herd: **MariaDB 10.11.6 :3306**, **PostgreSQL 18 :5432**, SQLite nativ; MySQL war nicht installiert — seit 28.10.2026 **MySQL 8.4.11 :3307** (R-33, Freigabe Hoang)	Herd `services list` 28.10.2026
Wie ist die Site erreichbar?	`https://phpdbadmin.test` (Herd: gesichert, PHP 8.5, Node 24); `APP_URL` in der `.env` steht noch auf `http://localhost:8000`, `APP_NAME=Laravel`, `APP_LOCALE=en`, App-Datenbank SQLite	Herd `site info`, `.env` (nur Schlüsselnamen gelesen)
Was kann Laravel 13 schon für mehrere Datenbanken?	Verbindungen zur Laufzeit: `DB::build()`, `DB::connectUsing()`; Introspektion für alle Treiber: `Schema::getSchemas()`, `getTables()`, `getViews()`, `getTypes()`, `getColumns()`, `getIndexes()`, `getForeignKeys()`; Vorschau: `Blueprint::toSql()`, `Connection::pretend()`	Quellcode `vendor/laravel/framework` 13.39
Was hat Basalto, das hier wiederverwendbar ist?	Starter-Kit-Bereinigung mit Passkey- und Code-Anmeldung (TB 00, Scheibe 1), Tests auf drei bzw. vier Datenbanken, Diagramm-Generator und PDF-Bau der Doku, Lektionen zu Fortify und `laravel/passkeys` (Routen-Übernahme bei Minor-Update, globale Bindung von `{passkey}`) — eigenes Projekt, Übernahme erlaubt	`~/Development/basalto`, Vault [[Nguyen Development]]
Wie steht der Markt?	phpMyAdmin 5.2.3 nur bis PHP 8.3 freigegeben, phpPgAdmin ruht, Adminer 6.2.0 aktiv, Jaxon DbAdmin 0.35 als neuer PHP-Mitbewerber	Marktbild
Ist der Name frei?	`phpdbadmin.ch` frei, `phpdbadmin.com` belegt seit 30.08.2025; keine gleichnamige Software gefunden	Whois und Websuche 28.10.2026

3. Rückfragen

A. Rahmen und Governance

Nr.	Frage	Annahme (gilt, solange nichts dagegen spricht)	Antwort/Entscheid
R-01	**Wer trägt das Vorhaben?** Eigenständiges Produkt der Einzelfirma Nguyen Development (wie Basalto), privates Werkzeug (wie Fristo als Nebenprodukt) oder ein Modul von Basalto?	Eigenständiges Produkt von Nguyen Development, zuerst für den eigenen Gebrauch	✅ 28.10.2026: **wie Annahme** — Produkt von Nguyen Development (zusammen mit R-40: MIT)
R-02	**Rollen:** alle DSDM-Rollen in Personalunion? Gibt es einen Gegenleser (Business Ambassador)?	Personalunion; Hoang ist erster Nutzer; ein Laravel-Entwickler aus dem Umfeld als Gegenleser ab Alpha	
R-03	**Vorrang und Zeit** gegenüber Basalto (Beta Anfang 2027) und Fristo (Pilot Frühling 2027)	Basalto behält Vorrang bis zur Beta; phpdbadmin läuft in kleinen Timeboxes daneben, Stufen 0–1 zuerst	✅ 28.10.2026: **wie Annahme** — neben Basalto
R-04	**Name:** Arbeitstitel «phpdbadmin» behalten oder Namensfindung (wie bei Basalto)? Der Name ist nahe an phpMyAdmin und nennt Technik statt Nutzen	Arbeitstitel bis vor dem ersten öffentlichen Auftritt; Namensfindung als Prüfpunkt der Feasibility (F-10)	✅ 28.10.2026: **phpdbadmin ist der Name** (endgültig) — F-10 wird «Name absichern»: Markenrecherche, Domain `.ch`, eigenständiger Auftritt
R-05	**Arbeitsvertrag:** Gilt die Meldung der Nebenbeschäftigung wie bei Basalto auch hier?	Ja, Vorbedingung vor dem ersten öffentlichen Auftritt	
R-06	**Doku-Spiegel:** nur Repo und Vault?	Nur Repo und Vault-Projektnotiz	
R-07	**Repo:** GitHub `hoang-79/phpdbadmin`, privat?	Privat bis v1.0, dann nach Lizenzentscheid (R-40)	
R-08	**Branches und Auslieferung:** direkt auf `main`, gebündelte Pushes?	`main` lokal, Pushes gebündelt; Auslieferungsweg mit dem Betriebsmodell (R-11)	

B. Produkt und Positionierung

Nr.	Frage	Annahme	Antwort/Entscheid
R-10	**Zielgruppe zuerst:** Laravel-/PHP-Entwickler lokal, Betriebsteams auf Servern oder Fachleute mit Datenpflege?	Entwickler zuerst, Betriebsteams gleich danach (Rechte, Protokoll in v1.0); Fachleute profitieren von der Führung	
R-11	**Betriebsmodell:** (a) nur lokal (Herd, ein Benutzer) · (b) lokal und selbst gehostet auf einem Server (mehrere Konten) · (c) zusätzlich als SaaS	(b) — kein SaaS, weil dafür fremde Zugangsdaten und Netzwege auf unseren Servern lägen	✅ 28.10.2026: **wie Annahme** — lokal und selbst gehostet
R-12	**Mehrere Konten ab v1.0?**	Ja: Einladung, Rollen, Rechte je Verbindung (A-06, A-07)	

Nr.	Frage	Annahme	Antwort/Entscheid
R-13	**Was heisst «intuitiver» konkret** — gibt es Vorbilder, die dir gefallen (TablePlus, Beekeeper Studio, Tabellen-Editor von Supabase, Notion-Datenbanken)?	Arbeitsfläche wie TablePlus, Bearbeiten wie der Tabellen-Editor von Supabase, festgehalten in den Prinzipien P-01 bis P-11 und im Bedienkonzept	
R-14	**Weitere Datenbanken später** (SQL Server, Oracle, CockroachDB, ClickHouse, MongoDB, Redis)?	Nicht vor v1.2; die Dialekt-Schicht hält die Tür offen (SQL Server hat einen Laravel-Treiber)	
R-15	**Mindestversionen der Server**	MariaDB 10.6, MySQL 8.0, PostgreSQL 14, SQLite 3.35 — in der Feasibility prüfen	
R-16	**phpMyAdmin-Gewohnheiten** übernehmen (Reiter «Operationen», URL-Aufbau, Designer)?	Nein, eigene Struktur; Begriffe dort angelehnt, wo sie verbreitet sind	
R-17	Soll phpdbadmin seine **eigene App-Datenbank** als Verbindung zeigen dürfen?	Nein — sonst liessen sich Konten und Rechte am System vorbei ändern	

C. Funktionsumfang

Nr.	Frage	Annahme	Antwort/Entscheid
R-20	**Umfang v1.0** = Stufen 0–2 wie im Scope-Diagramm?	Ja	
R-21	Verwaltung der **Datenbank-Benutzer und Rechte** (GRANT) erst in v1.1?	Ja	
R-22	**Export-Formate** in v1.0: CSV, JSON, SQL; Excel später?	Ja	
R-23	**KI-Hilfe** für SQL: nur optional, eigener Schlüssel, nur das Schema wird gesendet, Stufe 4?	Ja	
R-24	**Laravel-Extras** (Migration aus Tabelle, Herd-Erkennung) erst in Stufe 4 oder früher als Unterscheidungsmerkmal?	Stufe 4	
R-25	Darf das **Datenraster waagrecht scrollen** (Ausnahme von «keine Scrollbalken»)?	Ja, nur dort: fixierte erste Spalte, Verlaufskante mit Chevron, Spaltenwahl	
R-26	**Rückgängig für Datenänderungen** (D-09) schon in v1.0?	Ja als Should, begrenzt auf eine Zeilenzahl je Änderung	

D. Technik und Betrieb

Nr.	Frage	Annahme	Antwort/Entscheid
R-30	**Umgebungen** ausser Dev: Testing, Demo, Produktion?	Bis v1.0 nur Dev; danach eine öffentliche Demo mit SQLite-Beispieldatenbanken (Ort offen: cyon wie Basalto oder Laravel Cloud)	
R-31	**App-Datenbank**	SQLite als Vorgabe, die anderen drei möglich (B-06)	
R-32	**Neue Pakete** für SQL-Editor (CodeMirror 6, MIT) und Raster-Virtualisierung (TanStack, MIT)	Probe in der Feasibility (`composer` / `npm` trocken), Entscheid per ADR, Freigabe durch Hoang	✅ 28.10.2026: probeweise installiert (Freigabe Hoang); Entscheid in ADR-005/006

Nr.	Frage	Annahme	Antwort/Entscheid
R-33	**MySQL in Herd** installieren (8.4, Port 3307) für die Testmatrix	Ja, in TB 00	✅ 28.10.2026: in der Feasibility installiert (Herd meldete einen Fehler, legte den Dienst aber an)
R-34	**Mail:** lokal Herd-Mail, sonst SMTP der Installation	Ja	
R-35	**Bezeichner** im Code und in der App-Datenbank	Englisch (wie Fristo und Basalto), Oberfläche Deutsch und Englisch	
R-36	**Zeitzone**	Speichern in UTC, Anzeige in Europe/Zurich bzw. der Zeitzone des Kontos	

E. Geschäft

Nr.	Frage	Annahme	Antwort/Entscheid
R-40	**Lizenz:** Open Source (MIT), Open Core oder kommerziell?	Open Source (MIT); Team-Funktionen später allenfalls als bezahlter Zusatz	✅ 28.10.2026: **wie Annahme** — MIT
R-41	**Vertrieb:** Git und `composer create-project`; Docker-Image?	Git und Composer ab v1.0, Docker-Image als Could	
R-42	**Bezug zu Basalto:** eigenständig oder später ein Basalto-Modul «Datenbank»?	Eigenständig; gemeinsamer Code nur über Pakete	

4. Entscheide aus der interaktiven Runde vom 28.10.2026

Nr.	Gefragt	Gewählt	Folge
R-01, R-40	Träger und Lizenz	Nguyen Development, MIT (Empfehlung)	Lizenzdatei MIT vor dem ersten öffentlichen Push; Abhängigkeiten MIT-verträglich (F-9); kein GPL-Code
R-11	Betriebsmodell	lokal und selbst gehostet (Empfehlung)	Konten, Einladung, Rechte je Verbindung ab v1.0; Hosts-Liste; Installationsweg für Server (B-05)
R-03	Vorrang	neben Basalto (Empfehlung)	kleine Timeboxes, Stufen 0–1 bis zur Alpha zuerst
R-04	Name	**phpdbadmin ist der Name** (statt Arbeitstitel mit späterer Namensfindung)	`phpdbadmin.ch` sichern (Hoang); Markenrecherche zur Nähe zu «phpMyAdmin» und eigenständiger Auftritt vor dem ersten öffentlichen Auftritt (F-10, Risiko R6)

Neue Rückfragen ergeben sich daraus keine; die übrigen Fragen laufen mit ihrer Annahme weiter.

Anhang C — Ideen-Backlog — phpdbadmin

Stand: 28.10.2026 · erste Sammlung. Alle Ideen mit Nutzen, Herkunft und MoSCoW-Vorschlag. Was in CR-001 § 4 als Anforderung steht, trägt deren Nummer; die übrigen sind Kandidaten für spätere Change Requests. Herkunft: **H** Hoangs Auftrag · **M** Marktbild · **F** Funktionsraster · **L** Lektionen aus Basalto und Fristo · **C** Vorschlag Claude.

1. Anmeldung und Konten

Idee	Nutzen	Herkunft	MoSCoW	Bezug
Anmeldung nur mit Passkey oder Code per Mail	kein Passwort, das man verlieren oder erraten kann	H	Must	A-01 bis A-03
Passkey nach der ersten Code-Anmeldung anbieten	schnellere Anmeldung im Alltag	C	Should	A-03
Einladung mit Ablauf statt Registrierung	keine fremden Konten	C	Must	A-06
Rechte je Verbindung (lesen, schreiben, verwalten)	Teams ohne geteilte DB-Passwörter	M	Should	A-07
Sitzungen sehen und beenden	verlorenes Gerät sperren	C	Could	A-08

2. Verbindungen

Idee	Nutzen	Herkunft	MoSCoW	Bezug
Umgebungsfarbe und Schreibschutz für die Produktion	Verwechslung lokal/Produktion verhindern	C	Must	V-04
Rotes Favicon und Präfix im Seitentitel bei Produktion	Gefahr auch im Browser-Tab sichtbar	C	Should	V-04
«Passwort nicht speichern» je Verbindung	heikle Server ohne gespeicherte Zugangsdaten	C	Should	V-03
Erlaubte Hosts und Ordner	kein Sprungbrett ins interne Netz	L	Must	V-05
Herd-Dienste und `.env` der Herd-Sites erkennen	Verbindung mit einem Klick	C	Could	V-07
SSH-Tunnel	interne Server erreichen	F	Could	V-08
Verbindungsliste aus phpMyAdmin-Konfiguration übernehmen	Umstieg erleichtern	C	Won't	—

3. Bedienung

Idee	Nutzen	Herkunft	MoSCoW	Bezug
Befehlspalette ⌘K	alles mit der Tastatur erreichen	C	Must	O-02
Reiter, die das Neuladen überleben	Arbeit nicht verlieren	C	Should	O-03
Zeilen-Detail als Seitenschublade mit allen Feldern und Beziehungen	lange Werte und Bezüge auf einen Blick	C	Should	D-05

Idee	Nutzen	Herkunft	MoSCoW	Bezug
Karten statt Tabelle auf dem Telefon	lesbar ohne waagrechtes Scrollen	L	Could	O-07
Kürzel-Übersicht mit `?`	Tastatur lernen	C	Should	O-05

4. Daten

Idee	Nutzen	Herkunft	MoSCoW	Bezug
Änderungen sammeln, SQL-Vorschau, einmal anwenden	nichts passiert ungesehen	H, C	Must	D-04
Rückgängig für angewendete Datenänderungen	Fehler in Sekunden beheben	C	Should	D-09
Betroffene Zeilen vor UPDATE und DELETE zählen	Ausmass vor dem Ausführen sehen	C	Must	Q-07
Fremdschlüssel vorwärts und rückwärts folgen	Zusammenhänge ohne SQL	F	Should	D-03
Spaltenstatistik (Füllgrad, NULL-Anteil, kleinster und grösster Wert, häufigste Werte)	Datenqualität messen statt Bildschirme ansehen	L	Could	—
Abfrage-Parameter (`:von`, `:bis`) mit Formular	gespeicherte Abfragen wiederverwenden	C	Could	Q-04
Vergleich zweier Abfrageergebnisse	«was hat sich geändert?»	C	Could	—
Testdaten erzeugen (Faker) für eine Tabelle	lokale Entwicklung	C	Could	—
Datenmaskierung für Bildschirmfreigaben (E-Mail, Telefon)	Vorführen ohne Datenschutzproblem	M	Could	—

5. Struktur und SQL

Idee	Nutzen	Herkunft	MoSCoW	Bezug
SQL-Vorschau für jede Strukturänderung	DDL verstehen und prüfen	H, C	Must	S-02, S-03
Gesundheitsprüfung: Fremdschlüssel ohne Index, Tabellen ohne Primärschlüssel, `TEXT` an der 64-KB-Grenze, `timestamp` ohne Vorgabe	Fallen aus Basalto früh sehen	L	Could	—
Laravel-Konventionen erkennen (`migrations`, `deleted_at`, `created_at`) — Filter «ohne gelöschte»	Laravel-Daten richtig lesen	C	Could	—
Laravel-Migration aus Tabelle	Bestehendes in Code überführen	C	Could	S-09
Schemavergleich mit Änderungsskript	Stände angleichen	F	Could	S-08
Beziehungsdiagramm	Datenmodell verstehen	F	Should	S-07
EXPLAIN als Baum mit Hinweisen	langsame Abfragen verstehen	F	Should	Q-06
SQL-Hilfe mit KI (eigener Schlüssel, nur Schema)	Abfragen schneller schreiben	M	Could	Q-08
«Erkläre diese Tabelle» mit KI	fremde Datenbanken verstehen	C	Won't (vorerst)	—

6. Export, Import, Betrieb

Idee	Nutzen	Herkunft	MoSCoW	Bezug
Export CSV, JSON, SQL im Hintergrund	grosse Tabellen ohne Zeitüberschreitung	F	Must	E-01
CSV-Import mit Spaltenzuordnung und Probelauf	Fehler vor dem Schreiben sehen	F, L	Must	E-02
Kopieren als TSV, Markdown, INSERT	schnell in Ticket oder Doku	C	Should	E-05
Sicherung mit `mariadb-dump`, `mysqldump`, `pg_dump`	Rückweg vor grossen Änderungen	F	Should	E-03
Protokoll aller schreibenden Aktionen	nachvollziehen, wer was geändert hat	M	Must	B-04
Docker-Image	Betrieb neben Datenbanken in Containern	M	Could	R-41
Öffentliche Demo mit SQLite-Beispieldatenbanken	ausprobieren ohne Installation	C	Should	R-30

Architektur-Hinweis

Fast jede Idee berührt die **Dialekt-Schicht**: Was eine Funktion je Datenbank anders tun muss, gehört dorthin und bekommt einen Vertragstest, der gegen alle vier Treiber läuft. Ideen, die nur auf einem Treiber sinnvoll sind (Events in MySQL, Sequenzen in PostgreSQL, `PRAGMA` in SQLite), erscheinen nur dort — mit derselben Bedienung.